```
Customer c;
c = database.findCustomer("dave");    /* Java */
```

This fragment declares the variable c to be of type Customer and sets it to reference the customer object for Dave that we've created from some database record. So, the type of the object in c is Customer, right?

Maybe. However, even in Java, the issue is slightly deeper. Java supports the concept of *interfaces*. An interface is a list of methods that are supported together by classes that implement the interface. A Java class can be declared as implementing multiple interfaces. More to the point, a variable can be declared as being typed to an interface, rather than a class. Using this facility, you may have defined your classes as follows:

```
public interface Customer {
  long  getID();
  Calendar getDateOfLastContact();
  // ...
}

public class Person implements Customer {
  public long getID() { ... }
  public Calendar getDateOfLastContact() { ... }
  // ...
}
```

So, even in an explicitly typed language, the class is not always the type—sometimes the type is a subset of the class, and sometimes objects implement multiple types.

In Ruby, the class is never (well, almost never) the type. Instead, the type of an object is defined by what messages it responds to. The idea that typing is implicitly based on messages defined rather than being explicitly declared is called *duck typing*. If an object walks like a duck and quacks like a duck, then the interpreter is happy to treat it as if it were a duck, even if it is just a duck-shaped puppet.

Earlier in the book, we said that a type defined both the values that could be assigned to a variable and the expected behavior of those values. In Ruby (when not using RBS or Sorbet), those values and behavior are both enforced only by the set of messages that are passed to the value when the program runs.

Let's look at an example to see why taking advantage of Ruby's dynamic nature can be helpful. Perhaps we've written a method to write our customer's name to the end of an open file:

ducktyping/add_customer.rb

```
class Customer
  def initialize(first_name, last_name)
    @first_name = first_name
    @last_name = last_name
  end

  def append_name_to_file(file)
    file << @first_name << " " << @last_name
  end
end
```